Poornima College of Engineering

Department of Information Technology

Computer Architecture and Organization

Unit 3_ Central Processing Unit

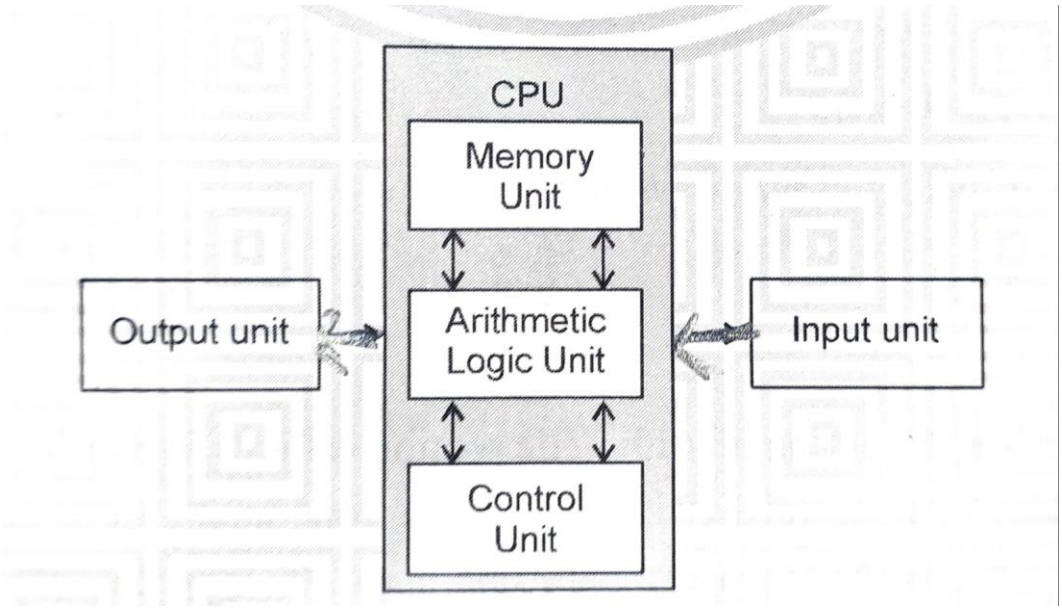
INTRODUCTION

A central processing unit (CPU) is the electronic circuitry within a computer that carries out the instructions of a computer program by performing the basic arithmetic, logic, controlling, and input/ output (1/0) operations specified by the instructions. The data and the instructions are temporarily stored in its memory unit. After performing operations, the result of operations can be stored in this memory unit.

The design of CPU involves choosing the hardware for implementing the machine instructions. A programmer using assembly language has to be aware of the register set, the memory structure, the type of data supported by the instructions, and the function of each instruction.

CPU itself has following three components:

1. Memory or Storage Unit
2. Control Unit
3. ALU (Arithmetic Logic Unit)



1. Memory or Storage Unit

This unit can store instructions, data, and intermediate results. This unit supplies information to other units of the computer when needed. It is also known as internal storage unit or the main memory or the primary storage or Random Access Memory (RAM). Its size affects speed, power, and capability. Primary memory and secondary memory are two types of memories in the computer.

Functions of the memory unit are

- It stores all the data and the instructions required for processing.
- It stores intermediate results of processing
- It stores the final results of processing before these results are released to an output device.
- All inputs and outputs are transmitted through the main memory.

2. Control Unit

This unit controls the operations of all parts of the computer but does not carry out any actual data processing operations. This unit also checks the correctness of sequence of operations. It fetches instructions in a program from the primary storage unit, interprets them and generates control signals to ensure correct execution of the program.

Functions of this unit are

- It is responsible for controlling the transfer of data and instructions among other units of a computer.
- It manages and coordinates all the units of the computer.
- It obtains the instructions from the memory, interprets them, and directs the operation of the computer,
- It communicates with Input/Output devices for transfer of data or results from storage.

3. ALU (Arithmetic Logic Unit)

This unit consists of two subsections namely,

(i) Arithmetic Section

(ii) Logic Section

(i) Arithmetic Section: Function of arithmetic section is to perform arithmetic operations like addition, subtraction, multiplication, and division. All complex operations are done by making repetitive use of the above operations. The ALU performs the operations and the result is stored in an internal memory unit of the CPU.

(ii) Logic Section: Function of logic section is to perform logic operations such as OR, AND, invert, exclusive - OR on binary words, comparing, selecting, matching, and merging of data.

Some other components of CPU:

Registers, a memory location within the actual processor that work at very fast speeds. It stores instructions which await to be decoded or executed.

1. PC-program counter - stores address of the current instruction being executed

2. MAR-memory address register - stores the address of the current instruction being executed

3. MDR-memory data register - it is temporary storage component in CPU. It allows to store the address of the data sent by the memory unit.

4. CIR-current instruction register-stores actual instruction that is being decoded and executed

5. ACC-accumulator - stores result of calculations

Buses:

1. **Address bus:** carries the ADDRESS of the instruction or data
2. **Data bus:** carries data between processor and the memory
3. **Control bus:** sends control signals such as: memory read, memory write.

GENERAL REGISTER ORGANIZATION

When a large number of registers are included in the CPU, it is most efficient to connect them through a common bus system. The registers communicate with each other not only for direct data transfers, but also while performing various microoperations. Hence it is necessary to provide a common unit that can perform all the arithmetic, logic, and shift microoperations in the processor.

A bus organization for seven CPU registers is shown in Fig. The output of each register is connected to two multiplexers (MUX) to form the two buses A and B. The selection lines in each multiplexer select one register or the input data for the particular bus. The A and B buses form the inputs to a common arithmetic logic unit (ALU). The operation selected in the ALU determines the arithmetic or logic microoperation that is to be performed. The result of the microoperation is available for output data and also goes into the inputs of all the registers. The register that receives the information from the output bus is selected by a decoder. The decoder activates one of the register load inputs, thus providing a transfer path between the data in the output bus and the inputs of the selected destination register. The control unit that operates the CPU bus system directs the information flow through the registers and ALU by selecting the various components in the system.

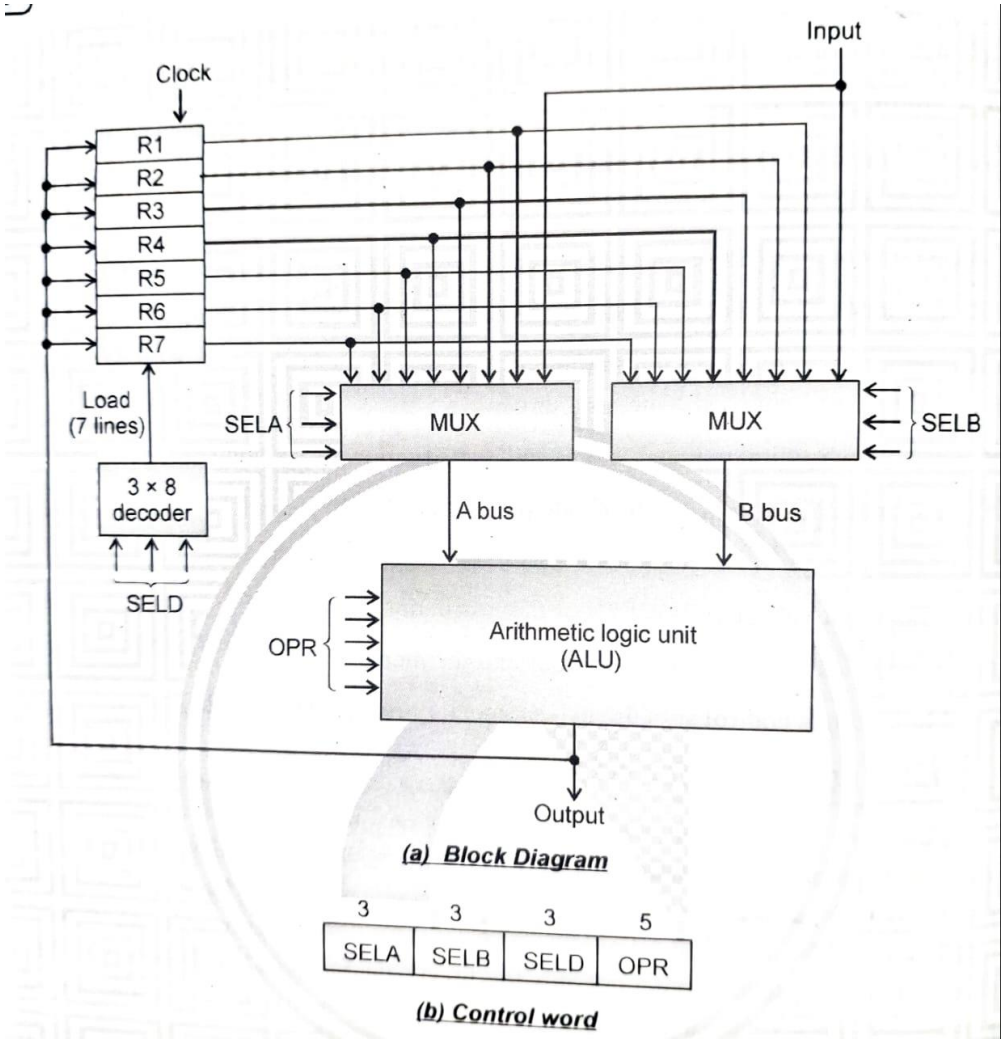


Fig.: Register set with common ALU

For example, to perform the operation.

$$\boldsymbol{R1 \leftarrow R2 + R3}$$

The control must provide binary selection variables to the following selector inputs:

1. MUX A selector (SELA): To place the content of R2 into bus A.
2. MUX B selector (SELB): To place the content of R3 into bus B.
3. ALU operation selector (OPR): To provide the arithmetic addition **A+B**.
4. Decoder destination selector (SELD): To transfer the content of the output bus into R1.

The four control selection variables are generated in the control unit and must be available at the beginning of a clock cycle. The data from the two source registers propagate through the gates in

the multiplexers and the ALU, to the output bus, and into the inputs of the destination register, all during the clock cycle interval. Then, when the next clock transition occurs, the binary information from the output bus is transferred into R1. To achieve a fast response time, the ALU is constructed with high-speed circuits. The buses are implemented with multiplexers or three-state gates.

Control Word

There are 14 binary selection inputs in the unit, and their combined value specifies a control word. The 14-bit control word is defined in Fig. It consists of four fields. Three fields contain three bits each, and one field has five bits. The three bits of SELA select a source register for the A input of the ALU. The three bits of SELB select a register for the B input of the ALU. The three bits of SELD select a destination register using the decoder and its seven load output. The five bits of OPR select one of the operations in the ALU. The 14-bit control word when applied to the selection inputs specify a particular microoperation.

The encoding of the register selections is specified in Table.

Table: Encoding of register selection fields

Binary Code	SELA	SELB	SELD
000	Input	Input	None
001	R1	R1	R1
010	R2	R2	R2
011	R3	R3	R3
100	R4	R4	R4
101	R5	R5	R5
110	R6	R6	R6
111	R7	R7	R7

The 3-bitbinary code listed in the first column of the table specifies the binary code for each of the three fields. The register selected by fields SELA, SELB, and SELD is the one whose decimal number is equivalent to the binary number in the code. When SELA or SELB is 000, the corresponding multiplexer selects the external input data. When SELD=000, no destination register is selected but the contents of the output bus are available in the external output. The ALU provides arithmetic and logic operations. In addition, the CPU must provide shift operations.

Example of Micro-operations

A control word of 14 bits is needed to specify a microoperation in the CPU. The control word for a given microoperation can be derived from the selection variables. For example, the subtract microoperation given by the statement.

$$\boldsymbol{R1 \leftarrow R2 - R3}$$

Here R2 specifies for the A input of the ALU, R3 for the B input of the ALU. R1 for destination register, and an ALU operation to subtract A-B.

The control word for this microperation and a few others are listed in below table.

Table: Example of microperations of the CPU

Symbolic Designation					
Micro-operation	**SELA**	**SELB**	**SELD**	**OPR**	**Control word**
R1 ← R2 – R3	R2	R3	R1	SUB	010 011 001 00101
R4 ← R4 ∨ R3	R4	R5	R4	OR	100 101 100 01010
R6 ← R6 + 1	R6	—	R6	INCA	110 000 110 00001
R7← R1	R1	—	R7	TSFA	001 000 111 00000
Output ← R2	R2	—	None	TSFA	010 000 000 00000
Output ← Input	Input	—	None	TSFA	000 000 000 00000
R4 ← sh1 R4	R4	—	R4	SHLA	100 000 100 11000
R5 ← 0	R5	R5	R5	XOR	101 101 101 01100

Table: Operation of ALU

Operation select					Operation	Function
S3	S2	S1	S0	Cin		
0	0	0	0	0	F = A	Transfer A
0	0	0	0	1	F = A + 1	Increment A
0	0	0	1	0	F = A + B	Addition
0	0	0	1	1	F = A + B + 1	Add with carry
0	0	1	0	0	F = A + $\overline{B}$	Subtract with borrow
0	0	1	0	1	F = A+ $\overline{B}$+1	Subtraction
0	0	1	1	0	F = A - 1	Decrement A
0	0	1	1	1	F = A	Transfer A
0	1	0	0	×	F = A $\wedge$ B	AND
0	1	0	1	×	F = A $\vee$ B	OR
0	1	1	0	×	F = A $\oplus$ B	XOR
0	1	1	1	×	F = $\overline{A}$	Compliment A
1	0	×	×	×	F = shr A	Shift right A into F
1	1	×	×	×	F = shl A	Shift left A into F

STACK ORGANIZATION

A stack is a storage device for storing information in such a manner that the item stored **last is the first item retrieved** (**LIFO-last-in, first-out**). The stack is a memory unit with an address register called a stack pointer (SP), which always points at the top item in the stack. The two operations of a stack are the **insertion and deletion of items**.

The operation of insertion is called push, it is pushing the new item on the top of stack and the operation of deletion of existing item is called pop, push-operation increments the SP and pop-operation decrements the SP. Hence, all assembly programs should allocate memory for a stack.

- **Register Stack**

A stack can be placed in portion of a large memory or it can be organized as a collection of a finite number of memory words or registers. Figure shows below the organization of a 64-word register stack.

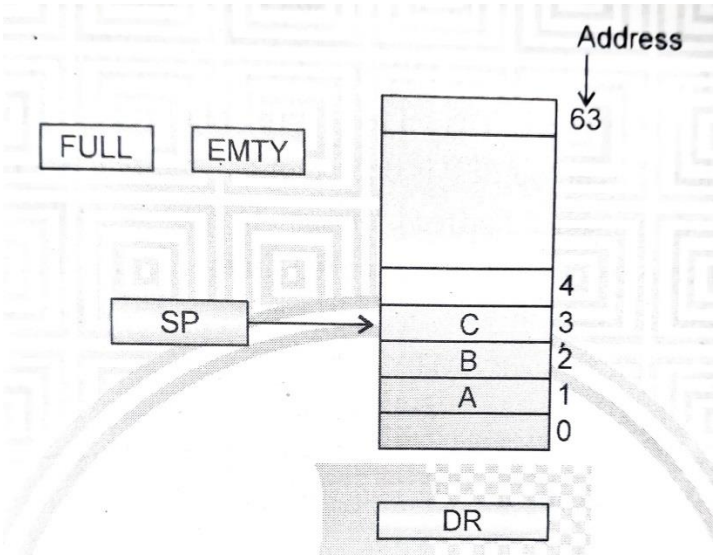


Fig: 64-word register stack

The stack pointer register SP contains a binary number whose value is equal to the address of the word that is currently on top of the stack. Three items are placed in the stack: A, B, and C, in that order.

Item C is on top of the stack so that the content of SP is now at address 3. To remove the top item, the stack is popped by reading the memory word at address 3 and decrementing the content of SP.

Item B is now on top the stack since SP holds address 2. To insert a new item, the stack is pushed by incrementing SP and writing a word in the next-higher location in the stack. Note that item C has been read out but not physically removed. This does not matter because when the stack is pushed, a new item is written in its place.

Initially, SP is cleared to 0, EMTY is set to 1, and FULL is cleared to 0, so that SP points to the word at address 0 and the stack is marked empty and not full. If the stack is not full (if FULL = 0), a new item is inserted with a push operation.

The push operation is implemented with the following sequence of microoperations:

SP ← SP + 1 ;	**Increment stack pointer**
M [SP] ← DR ;	**Write item on top of the stack**
If (SP= 0) then (FULL ← 1) ;	**Check if stack is full**
EMTY ← 0 ;	**Mark the stack not empty**

The stack pointer is incremented so that it points to the address of the next-higher word. A memory write operation inserts the word form DR into the top of the stack. Note that SP holds the address of the top of the stack and that M[SP] denotes the memory word specified by the address presently available in SP. The first item stored in the stack is at address 1. The last item is stored at address 0. If SP reaches 0, the stack is full of items so FULL is set to 1. This condition is reached if the top item prior to the last push was in location 63 and, after incrementing SP, the last item is stored in location 0. Once an item is stored in location 0, there are no more empty registers in the stack. If an item is written in the stack, obviously the stack cannot be empty, so EMTY is cleared to 0.

A new item is deleted from the stack if the stack is not empty (if EMTY=0). The pop operation consists of the following sequence of microoperations:

$\boldsymbol{DR \leftarrow M\,[SP]}$**;**	**Read item from the top of stack**
$\boldsymbol{SP \leftarrow SP - 1}$**;**	**Decrement stack pointer**
If (SP=0) then (EMTY←1);	**Check if stack is empty**
FULL←0;	**Mark the stack not full**

The top item is read from the stack into DR. the stack pointer is then decremented. If its value reaches zero, the stack is empty, so EMTY is set to 1. This condition is reached if the item read was in location 1. Once this item is read out, SP is decremented and reaches the value 0, which is the initial value of SP. Note that if a pop operation reads the item from location 0 and then SP is decremented, SP changes to 111111, which is equivalent to decimal 63. In this configuration, the word in address 0 receives the last item in the stack. Note also that an erroneous operation will result if the stack is pushed when FULL = 1 or popped when EMTY = 1.

- **Memory Stack**

The implementation of a stack in the CPU is done by assigning a portion of memory to a stack operation and using a processor register as a stack pointer. In the Figure shows a portion of computer memory partitioned into three segments: **program, data, and stack**.

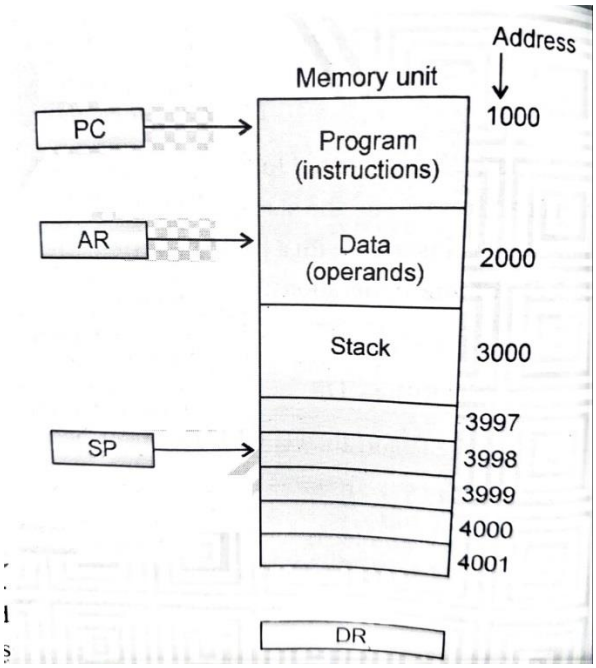


Fig: computer memory with program, data, and stack

The program counter PC points at the address of the next instruction in the program. The address register AR points at an array of data. The stack pointer SP points at the top of the stack. The three registers are connected to a common address bus, and either one can provide an address for memory. **PC is used during the fetch phase to read an instruction. AR is used during the execute phase to read an operand. SP is used to push or pop items into or from the stack.**

- **Reverse Polish Notation (Postfix Expressions)**

The common mathematical method of writing arithmetic expressions imposes difficulties when evaluated an arithmetic expressions by a computer. Arithmetic operations are usually written in **infix notation**: each operator resides between the operands (Ex: A+B).

Consider the simple arithmetic expression

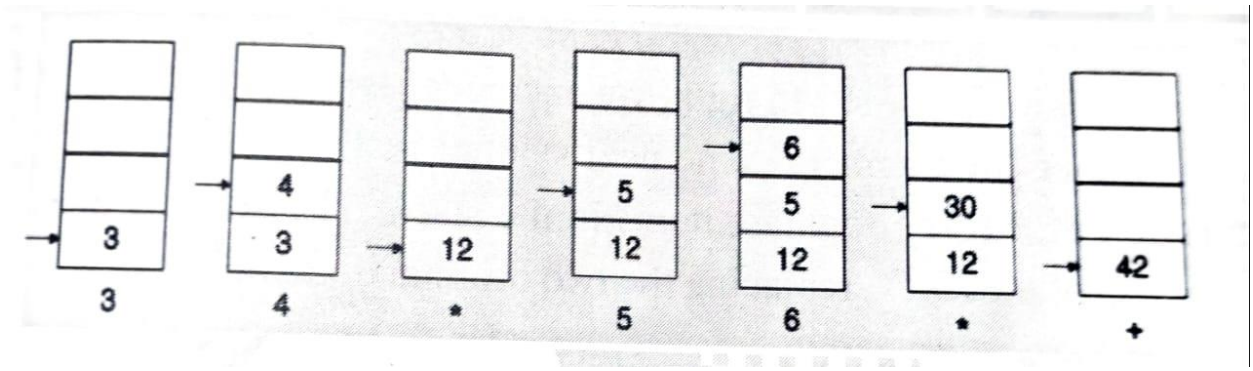
A*B + C*D

= AB*CD*+ ; **Reverse Polish Notation** (RPN)

The star (denoting multiplication) is placed between two operands A and B or C and D. The plus is between the two products. To evaluate this arithmetic expression it is necessary to compute the product A*B, store this product while computing C*D, and then sum the two products.

Example: (3 * 4) + (5 * 6)

= 34 *56 *+



Arithmetic expressions can be presented in prefix notation, operators are placed before the operands.

The postfix notation (reverse Polish notation) places the operator after the operand.

They are represent as the following-

A+B, Infix notation

+AB , Prefix notation

AB+ , Reverse postfix notation (RPN)

Note: The reverse Polish notation suits of stack manipulation.

E.g. the expression A * B + C * D is written in RPN (Reverse postfix notation) as AB*C D* + and is evaluated by scanning from left to right. When operator is found, the operation is performed by using operands on the left side of the operator. The operator and operands are replaced by the result of operation. The scan is continued and the procedure is repeated for every operator.

Evaluation of Arithmetic Expressions

Reverse Polish notation, combined with a stack arrangement of registers, is the most efficient way for evaluating arithmetic expressions. The stack is particularly useful for handling long, complex problems involving chain calculations. It is based on the fact that any arithmetic expression can be expressed in parentheses-free Polish notation.

The procedure consists of first converting the arithmetic expression into its equivalent reverse Polish notation. The operands are pushed into the stack in the same order. The operation to be performed as below-

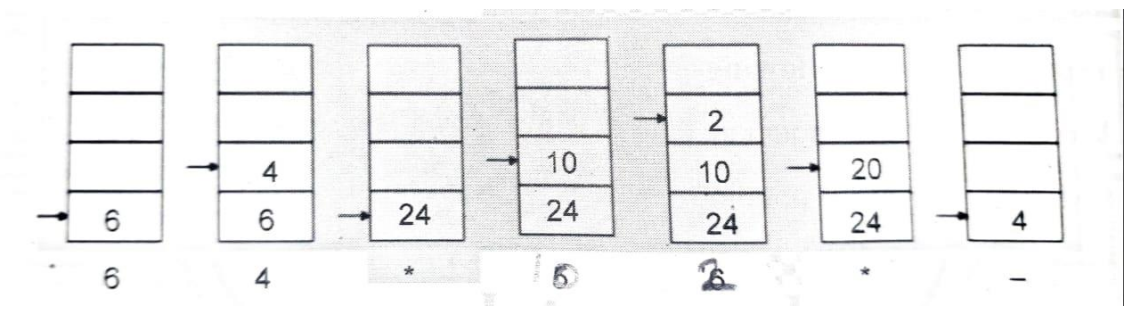
1. The two topmost operands in the stack are used for the operation, and
2. The stack is popped and the result of the operation replaces the lower operand. By pushing the operands into the stack continuously and performing the operations as defined above, the expression is evaluated in the proper order and the final result remains on top of the stack.

Example 3.1: Convert the following arithmetic expression into Reverse polish notation and show the stack operations for evaluating the expression result (6 * 4) - (10 * 2)

Solution: In reverse Polish notation, it is expressed as 64 * 10 2*-

Scanning the expression from left to right, we encounter two operands. First the number 6 is pushed into the stack, then the number 4. The next symbol is the multiplication operator. This causes a multiplication of the two topmost items in the stack. The stack is then popped and the product is placed on top of the stack, replacing the two original operands.

Next we encounter the two operands 10 and 2, so they are pushed into the stack. The stack operation that results from the next replaces these two numbers by their product. The last operation causes an arithmetic subtract of the two topmost numbers in the stack to produce the final result of 4.



Example 3.2: Convert the following arithmetic expressions from infix to Reverse Polish Notation (RPN).

(i) A * B + C * D + E * F

(ii) A * B + A*(B * D + C * E)

(iii) A + B * [C * D + E * (F + G)]

(iv) $\frac{A * [B + C * (D + E)]}{[F * (G + H)]}$

Solution:

(i) A * B + C * D + E * F

=(AB*) + (CD*) + (EF*)

$= (AB^* \; CD^* +) \; + \; (EF^* \;)$

$= AB^* \; CD^* + EF^* +$

(ii) $A * B + A * (B * D + C * E)$

$= A^*B + A^* (BD^* + CE^*)$

$= (AB^*) + A^*(BD^* \; CE^* +)$

$= (AB^* \;) + ABD^* \; CE^* + *$

$= AB^* \; ABD^* \; CE^* + * +$

(iii) $A + B * [C * D + E^* (F + G)]$

$= A + B^* [C * D + E * (FG+)]$

$= A + B * [C * D + FG + E^*]$

$= A + B * [(CD^*) + (FG + E^*]$

$= A + B * [CD * FG + E * +]$

$= A + CD * FG + E * + B *$

$= CD * FG + E * + B * A +$

(iv) $\dfrac{A * [B + C * (D + E)]}{[F * (G + H)]}$

$= \dfrac{A * [B + C * (DE +)]}{[F * (GH +)]}$

$= \dfrac{A * [B + DE + C *]}{GH + F *}$

$= \dfrac{A * [DE + C * B +]}{F * GH +}$

$= \dfrac{DE + C * B + A *}{F * GH +}$

$= DE + C * B + A * F * GH +/$

INSTRUCTION FORMATS

The CPU executes the instructions in a sequential manner if there is no interrupt and no branching instruction. It is the responsibility of the control unit to interpret the meaning of each instruction code format and provide necessary control signal whenever required. The instruction format can be divided into three fields as shown in fig.

Mode field	Operation code field	Address field

The address field of the instruction gives the address of the operands. Operands residing in memory are specified by their memory address and operands residing in the processor registers are specified with a register address.

The operation code field of the instruction is a group of bits that define various operations like addition, subtraction, shift etc. to be performed on the given operands. Each operation has its own separate code and that code is provided in the operation code field of the instruction.

Third field is the mode field. It specifies that how to find the address of the operands. This field gives that whether the instruction is using direct addressing or indirect addressing.

Instruction Classification:

The length of an instruction affects various factors like memory size, memory organization, bus architecture, CPU complexity and speed of the computer system.

The instruction set can be classified on the basis of available number of addresses in an instruction. There can be four types of instructions in an instruction set.

(i) Three address instructions

(ii) Two address instruction

(iii) One address instructions

(iv) Zero address instructions

(i) **Three address instructions** have 3-addresses in the address field of the instruction. It can use each address field to specify either a processor register or a memory operand.

The major advantage of this instruction format is that it reduces the program to a long extent. **Examples** of three address instructions are as follows

ADD	A, B, C;	M[A] ← M[B] + M[C]
MUL	R1, D, E;	R1 ← M[D] * M[E]

(ii) **Two address instruction** are most common instructions in computer systems. It has two addresses in the address fields of the instruction. In this type of instruction, first address in the instruction is assumed to be both source and the destination where the result of the operation is transferred.

Examples:

MOV	R1, X;	R1 ←M[X]
ADD	R2, Y;	R2 ←R2 + M[Y]

(iii) **One address instructions** are having only one address in the address field of the instruction. The accumulator is assumed as implied address. In this type of instructions, the operations are performed in the one address given and accumulator.

Examples:

LOAD	B	AC ←M[B]
ADD	C	AC ← AC + M[C]

(iv**) Zero address instructions** do not have address field in the instruction. These instructions use stack operation to perform various tasks. To evaluate arithmetic expressions in computer, it is necessary to convert the expression into **reverse polish notation**.

Examples:

PUSH A	M [SP+1]←A
ADD	M [SP + 1] ← (A+B)

Example: Explain the influence of the number of addresses on a computer program to evaluate the following arithmetic statement.

$$X = (A*B) + (C*D)$$

Use the symbols ADD, SUB, MUL and DIV for the four arithmetic operations: MOV for transfer operation; and LOAD and STORE for transfers to and from memory and AC register. Here A, B, C and D are memory addresses where operands are stored. Result must be stored in memory at address X.

Solution: The program that evaluate the given function is given below

Three-Address Instructions

MUL	R1, A, B	R1←M[A] * M[B]
MUL	R2, C, D	R2←M(C) * M[D]
ADD	X, R1, R2	X←R1+R2

Two-Address Instructions

MOV	R1, A	R1←M[A]
MUL	R1, B	R1←R1 * M[B]
MOV	R2, C	R2←M[C]
MUL	R2, D	R2←R2 * M[D]
ADD	R1, R2	R1←R1 + R2
MOV	X, R1	M[X]←R1

One-Address Instructions

LOAD	A	AC←M[A]
MUL	B	AC←AC * M[B]
STORE	T	M[T]←AC
LOAD	C	AC←M[AC]
MUL	D	AC←AC * M[D]
ADD	T	AC←AC + M[T]
STORE	X	M[X]←AC

Zero-Address Instructions

PUSH	A	SP←SP+1, M[SP]←A
PUSH	B	SP←SP+1, M[SP]←B
MUL		SP←SP+1, M[SP]←A * B
PUSH	C	SP←SP+1, M[SP]←C
PUSH	D	SP←SP+1, M[SP]←D
MUL		SP←SP+1, M[SP]←C * D
ADD		A * B + C * D
POP	X	M[X]←M[SP], SP←SP-1

ADDRESSING MODES

Addressing mode provides different ways for accessing an address to given data to a processor. The opcode (operation code) field of an instruction specifies the operation to be performed on some operands. These operands may be stored in processor registers or in memory words or may be provided directly. The way in which the operands are provided during execution is called addressing mode of that instruction. In other words, the addressing mode specifies how to find the effective address of an operand using the information available in instruction.

The most common addressing modes are:

(i) Implicit addressing mode

(ii) Immediate addressing mode

(iii) Direct addressing mode

(iv) Indirect addressing mode

(v) Register addressing mode

(vi) Register indirect addressing mode

(i) Implicit Addressing Mode: This addressing mode is also known as implied addressing mode. It does not explicitly specify an effective address for either source or the destination. Either the source (if any) or destination effective address is implied by the opcode itself.

For example, **"Set Carry Bit"** is an implicit mode instruction because the operand is carny bit in flag register, which is implicit in instruction. The instruction format for implicit addressing mode contains only opcode field. There is no operand field.

Opcode

Fig. Instruction format for implicit addressing mode

(ii) Immediate Addressing Mode: This is the simplest form of addressing. Here, the operand is given in the instruction itself. This mode is used to define a constant or set initial values of variables. In the immediate addressing mode, the instruction does not state explicitly the location of the operand, rather, it explicitly states the operand itself. The operand becomes an integral part of the instruction.

Opcode	Operand

Fig. Instruction format for Immediate Addressing Mode

(iii) Direct Addressing Mode: In direct addressing mode, effective address of the operand is given in the address field of the instruction. It requires one memory reference to read the operand from the given location and provides only a limited address space. The instruction contains the opcode and effective address of the operand.

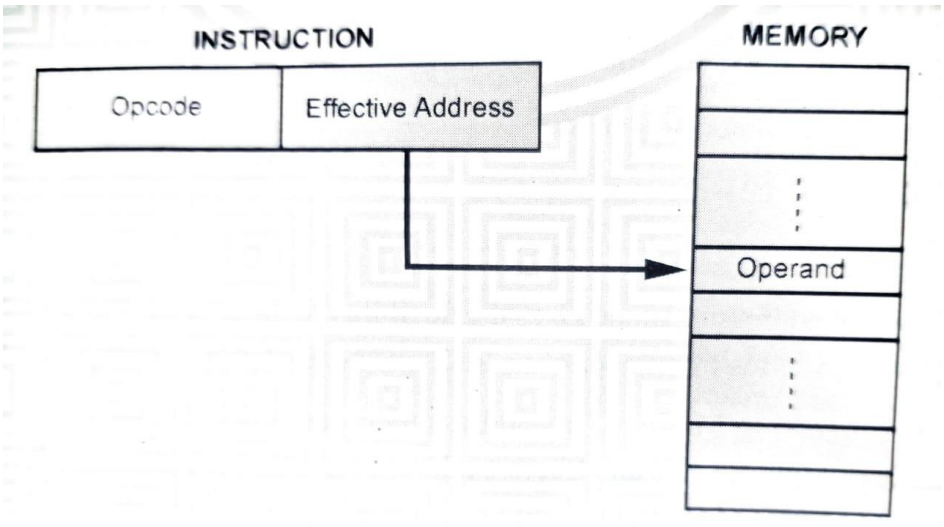


Fig. Direct addressing mode

(iv) Indirect Addressing Mode: In indirect addressing mode, the address field of the instruction provides the address where the effective address is stored in memory. **The advantage of this mode is that for the word length of N, an address space of 2^N can be addressed**. The disadvantage is that instruction execution requires two memory reference to fetch the operand.

Example of indirect addressing is shown in fig.

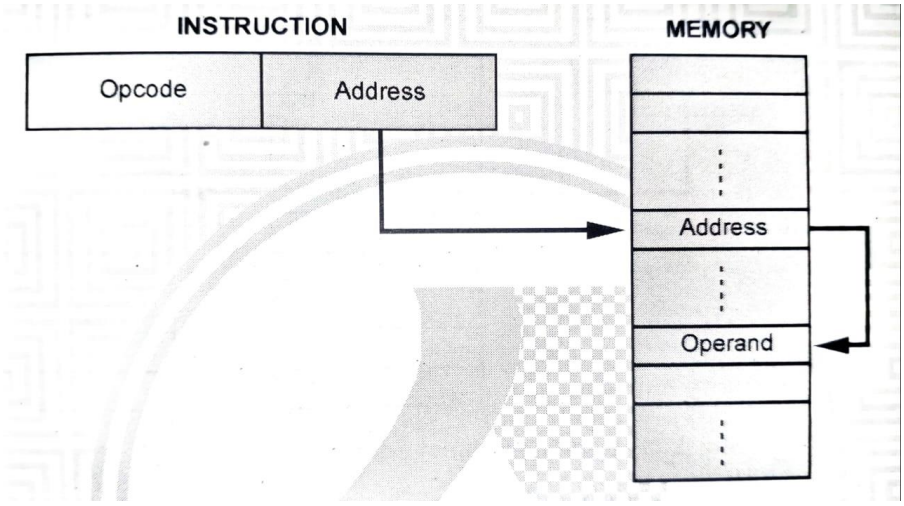


Fig. Indirect addressing mode

(v) Register Addressing Mode: Register addressing mode is similar to direct addressing. The only difference is that the address field of the instruction refers to a register rather than a memory location. **The advantage of register addressing is small address field is needed in the instruction**. In this mode the operands are in registers that reside within the processor unit. The disadvantage of register addressing is that the address space is very limited.

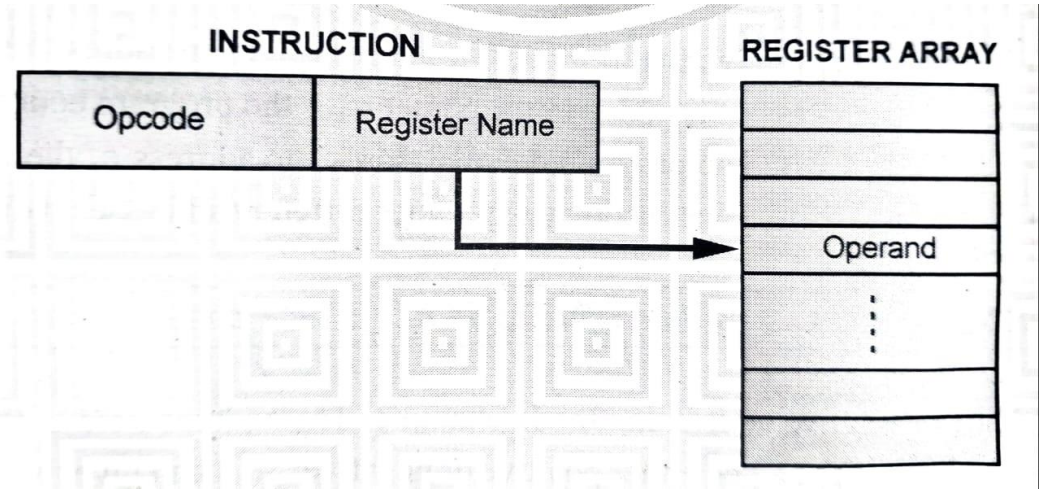


Fig. Register addressing mode

(vi) Register Indirect Addressing Mode: In this mode the instruction specifies a register in processor unit whose contents give the address of the operand in memory. The advantage of this mode is that address field of the instruction uses fewer bits to select a register than would have been required to specify a memory address directly.

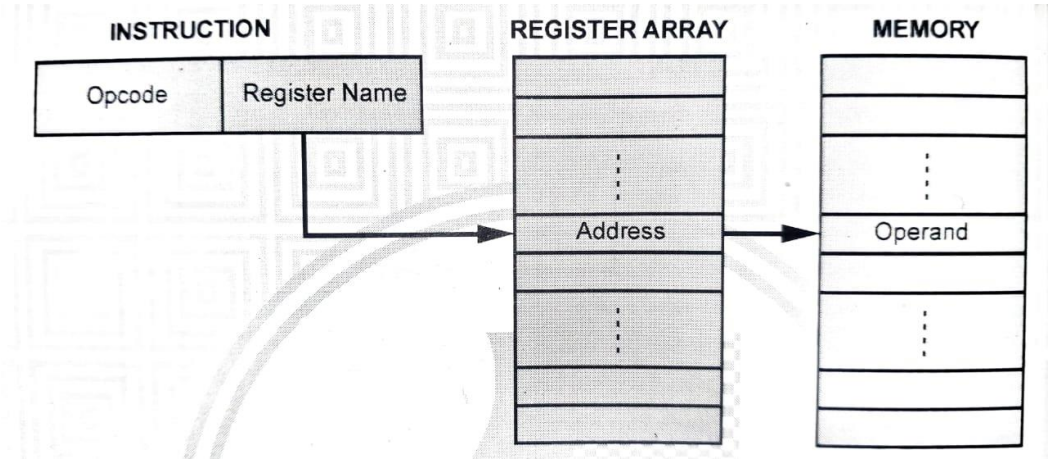


Fig. Register Indirect Addressing Mode

DATA TRANSFER AND MANIPULATION

Most of the computer systems have a common set of basic set of operations and corresponding instructions. However the opcode for these instructions may differ for same operation in different computer systems. We can classify computer instruction set in three categories.

(i) Data Transfer Instructions
(ii) Data Manipulation Instructions
(iii) Program Control Instructions

(i) Data Transfer Instructions: Data transfer instructions move the data from one location (source) to another location (destination) without changing the contents of the source location. The most common transfers are between memory and processor registers, between processor registers and input/output, and between the processor registers themselves.

Table: shows the most common data transfer instructions with their respective mnemonic and function used in many computer.

Name	Mnemonic	Function
Move	MOV	Copy the data from source location to destination location.
Store	ST	Store (save) the data from processor registers (mostly accumulator) to memory.
Load	LD	Load (copy) the data from memory to processor registers (mostly accumulator)
Input	IN	Transfers data from input terminal to processor registers (mostly accumulator)
Output	OUT	Transfers data from processor registers (mostly accumulator) to output terminal.
Push	PUSH	Write the data from processor registers to top of the stack.
Pop	POP	Read the data from top of the stack to processor registers.
Exchange	XCH	Exchange the data between two locations.

(ii) Data Manipulation Instructions: Data manipulation instruction perform operation on data and provide the capability of a computer system shows its computational capability. The data manipulation instructions can further be classified in three categories: (i) Arithmetic instruction (ii) Logical instruction and (iii) Shift instructions.

There are four basic arithmetic operations: Addition, subtraction, multiplication and division. However, some small computers have only addition and subtraction capabilities. Multiplication and divisions are performed through subroutines. Table: shows some of the arithmetic data manipulation instructions.

Logical instructions perform binary operations on strings of bits stored in processor registers. These instructions are useful for manipulating individual bits or a group of bits. The common logical operations are AND, OR, XOR and NOT.

In data manipulation instructions, there are four types of shift operations: (i) logical shift, (ii) arithmetical shift, (iii) rotate left shift, and (iv) rotate right shift. The related instructions are shown in table.

Table 3.4: Most common data manipulation instructions

Arithmetic		Logical		Shift	
Name	**Mnemonic**	**Name**	**Mnemonic**	**Name**	**Mnemonic**
Add	ADD	AND	AND	Logical shift right	SHR
Subtract	SUB	OR	OR	Logical shift left	SHL
Increment	INC	Exclusive OR	XOR	Arithmetic shift right	SHRA
Decrement	DEC	Complement	COM	Arithmetic shift left	SHLA
Multiply	MUL	Clear	CLR	Rotate right	ROR
Divide	DIV			Rotate left	ROL

PROGRAM CONTROL

Normal instructions are stored in successive memory locations and processor executes them in sequential manner. Each time, one instruction is fetched from memory, the program counter is incremented by one to indicate the address of the next instruction and then instruction is executed.

However, when some of the instructions are executed, they may change the address in the program counter. This results in altering the flow of control. Now processor fetches the instruction from new loaded address rather than the sequential address. This type of instructions are named as program control instructions.

Program control instructions specify conditions for altering the contents of the program counter whereas data transfer and manipulation instructions simply specify conditions for data processing operations. These program control instructions can be conditional and unconditional.

Table: Most common program control instruction

Name	Mnemonic	Function
Jump	JMP	Program control jumps to the given address may be outside the main program.
Branch	BR	Similar to jump but address is given by symbolic name.
Skip	SKP	Skip the next instruction if condition is met.
Call	CALL	Jump to the address of subroutine after saving the returning address.
Return	RET	Used with call instruction to return back to main program.

The conditional program control instructions indicate that flow of control will be changed if specified condition is met, otherwise sequential execution will be continued. Whereas the unconditional program control instructions do not check any condition and program control jumps to the new specified address. In addition to the most common program control instructions, there are two more instructions such as compare (CMP) and Test (TST). These two do not change the program sequence directly but affect the status bits which are used by other instructions. Status bits are stored in status register.

- **Status Bit Conditions**

It is sometimes convenient to supplement the ALU circuit in the CPU with a status register where status bit conditions can be stored for further analysis. Status bits are also called condition-code bits or flag bits. Figure below shows the block diagram of an 8-bit ALU with a 4-bit status register. The four status bits are symbolized by C, S, Z, and V. The bits are set or cleared as a result of an operation performed in the ALU.

1. Bit C (carry) is set to 1 if the end carry C_8 is 1. It is cleared to 0 if the carry is 0.
2. Bit S (sign) is set to 1 if the highest-order bit F_7 is 1. It is set to 0 if the bit is 0.
3. Bit Z (zero) is set to 1 if the output of the ALU contains all 0's. It is cleared to 0 otherwise. In other words, Z = 1 if the output is zero and Z = 0 if the output is not zero.
4. Bit V (overflow) is set to 1 if the exclusive-OR of the last two carries is equal to 1, and cleared to 0 otherwise. For the 8-bit ALU, V = 1 if the output is greater than +127 or less than -128.

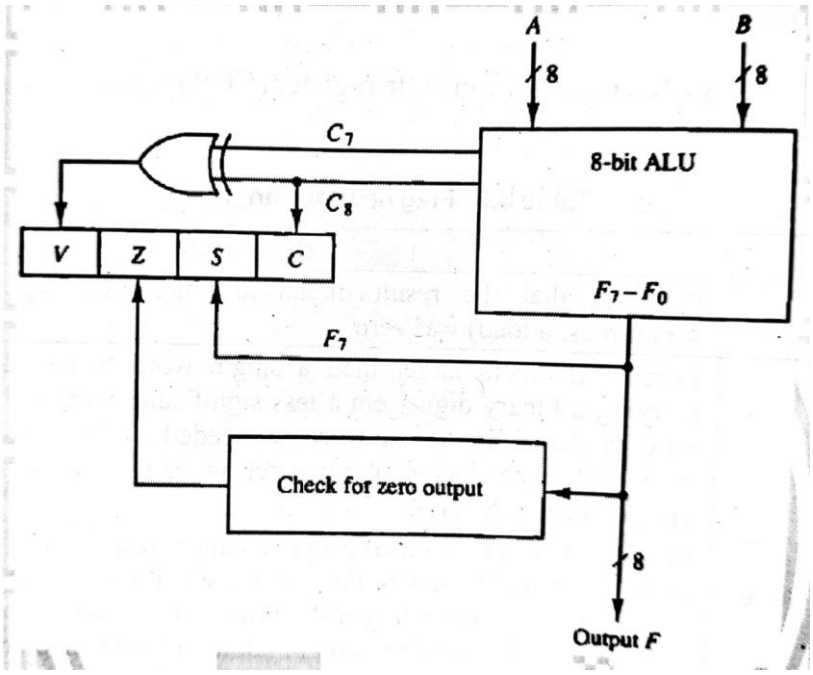


Fig: Status bit register

The status bits can be checked after an ALU operation to determine certain relationships that exist between the values of A and B. If bit V is set after the addition of two signed numbers, it indicates an overflow condition. If Z is set after an exclusive-OR operation, it indicates that A = B.

Example, let A =101x1100 where x is the bit to be checked. The AND operation of A with B = 00010000 produces a result 000x0000. If x = 0 the Z status bit is set, but if x = 1 the Z bit is cleared since the result is not zero.

- **Conditional Branch Instruction**

A conditional branch instruction is a branch instruction that may or may not cause a transfer of control depending on the value of stored bits in the PSR (processor status register).

Each conditional branch instruction tests a different combination of Status bits for a condition.

- If the condition is true, control is transferred to the effective address (PC ←Add).
- If the condition is false, the program continues with the next instruction (PC←PC+1)

Table: Conditional Branch Instructions

Mnemonic	**Brach condition**	**Tested condition**
BZ	Brach if zero	Z=0
BNZ	Brach if not zero	Z=1
BC	Brach if carry	C=1
BNC	Brach if no carry	C=0
BP	Brach if plus	S=0
BM	Brach if minus	S=1
BV	Brach if overflow	V=1
BNV	Brach if no overflow	V=0
Unsigned compare conditions (A - B)		
BHI	Brach if higher	A>B
BHE	Brach if higher or equal	A≥B
BLO	Brach if lower	A<B
BLOE	Brach if lower or equal	A≤B
BE	Brach if equal	A=B
BNE	Brach if no equal	A≠B
Signed compare conditions (A - B)		
BGT	Brach if greater than	A>B
BGE	Brach if greater or equal	A≥B
BLT	Brach if less than	A<B
BLE	Brach if less or equal	A≤B
BE	Brach if equal	A=B
BNE	Brach if not equal	A≠B

REDUCED INSTRUCTION SET COMPUTER (RISC)

A reduced instruction set computer (RISC) is a computer that uses a central processing unit (CPU) that implements the processor design principle of simplified instructions. **Reduced instruction actually means that the amount of work done by each instruction is reduced in terms of number of cycles-at most only a single data memory cycle and make them cheaper to design.** Dozens of cycles are required prior to completing the entire instruction. This results in faster processing.

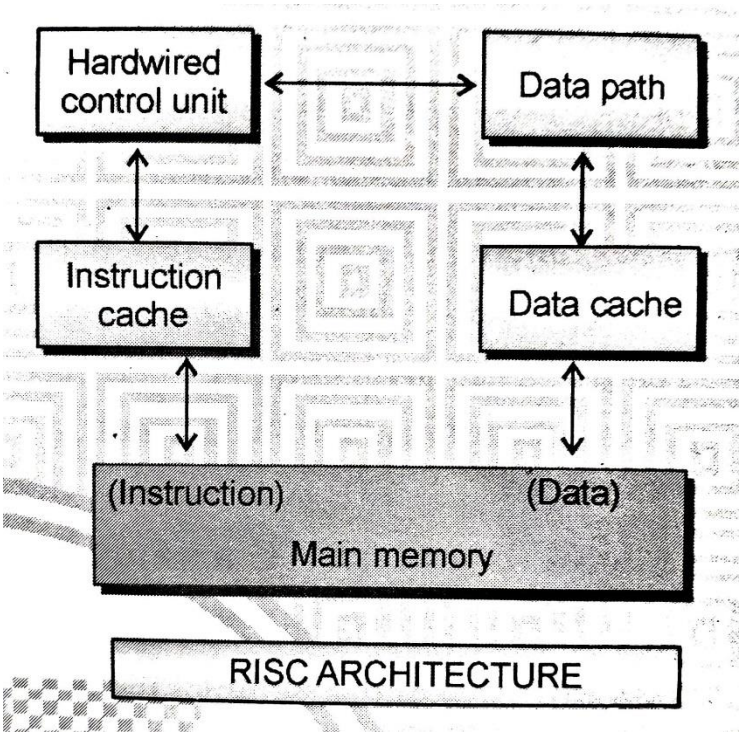


Fig: RISC architecture

The most basic RISC feature is a processor with a small core logic that allows engineers to increase the register set and increase internal parallelism.

Examples of RISC processors:

- IBM RS6000, MC88100
- DEC's Alpha 21064, 21164 and 21264 processors
- Alpha, ARC, ARM, AVR, MIPS, PA-RISC, PIC, Power Architecture, and SPARC.

Characteristic of RISC -

- Simpler instruction, hence simple instruction decoding.
- Instruction come under size of one word.
- Instruction take single clock cycle to get executed.
- More number of general purpose register.
- RISC permits any register to use in any context.

- Simple Addressing Modes.
- Less Data types.
- In RISC, more RAM is required to store assembly level instructions.
- RISC helps and supports few simple data types and synthesize complex data types.
- Used fixed length instructions for pipelining.
- Reduced instructions need a less number of transistors.

The Advantages of RISC architecture:

- RISC architecture has a set of instructions, so high-level language compilers can produce more efficient code.
- It allows freedom of using the space on microprocessors because of its simplicity.
- Many RISC processors use the registers for passing arguments and holding the local variables. RISC functions use only a few parameters, and the RISC processors cannot use the call instructions, and therefore, use a fixed length instruction which is easy to pipeline.
- The speed of the operation can be maximized and the execution time can be minimized. Very less number of instructional formats, a few numbers of instructions and a few addressing modes are needed.

The Disadvantages of RISC architecture:

- Mostly, the performance of the RISC processors depends on the programmer or compiler as the knowledge of the compiler plays a vital role while changing the CISC code to a RISC code
- While rearranging the CISC code to a RISC code, termed as a code expansion, will increase the size. And, the quality of this code expansion will again depend on the compiler, and also on the machine's instruction set.

COMPLEX INSTRUCTION SET COMPUTER (CISC)

A complex instruction set computer is a computer where single instructions can perform numerous low-level operations like a load from memory, an arithmetic operation, and a memory store or are accomplished by multi-step processes or addressing modes in single instructions, as its name proposes "Complex Instruction Set. The CISC architecture are designed to decrease the memory cost. Because, the large programs need more storage, thus increasing the memory cost and large memory becomes more expensive. To solve these problems, the number of instructions per program can be reduced by embedding the number of operations in a single instruction, thereby making the instructions more complex.

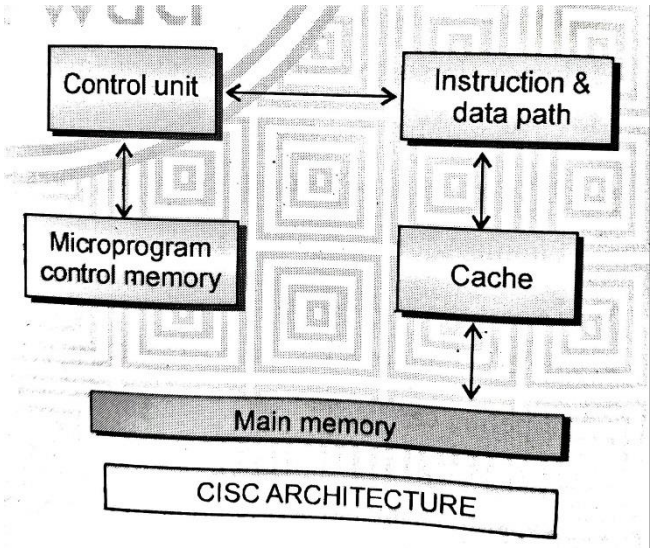


Fig: CISC architecture

Examples of CISC PROCESSORS:

- **IBM 370/168-**It was introduced in the year 1970. CISC design is a 32 bit processor and four 64-bit floating point registers.
- **VAX 11/780-**CISC design is a 32-bit processor and it supports many numbers of addressing modes and machine instructions which is from Digital Equipment Corporation.
- **Intel 80486-**It was launched in the year 1989 and it is a CISC processor, which has instructions varying lengths from 1 to 11 and it will have 235 instructions.

The Performance Equation:

The following equation is commonly used for expressing a computer's performance ability:

$$\frac{time}{program} = \frac{time}{cycle} \times \frac{cycles}{instruction} \times \frac{instructions}{program}$$

The CISC approach attempts to minimize the number of instructions per program, sacrificing the number of cycles per instruction. RISC does the opposite, reducing the cycles per instruction at the cost of the number of instructions per program.

Characteristic of CISC:

- Complex instruction, hence complex instruction decoding.
- Instruction are larger than one word size.
- Instruction may take more than single clock cycle to get executed.
- Less number of general purpose register as operation get performed in memory itself.
- Complex Addressing Modes

Advantages of CISC architecture:

- Microprogramming is easy: assembly language to implement, and less expensive than hard wiring a control unit.
- The ease of microcoding: new instructions allowed designers to make CISC machines upwardly compatible:
- Fewer instructions could be used to implement a given task.

Disadvantages of CISC architecture

- The performance of the machine slows down due to the amount of clock time taken by different instructions will be dissimilar
- Only 20% of the existing instructions is used in a typical programming event, even though there are various specialized instructions in reality which are not even used frequently.

Table: Difference between CISC and RISC

Architectural Characteristics	Complex Instruction Set Computer(CISC)	Reduced Instruction Set Computer(RISC)
Instruction size and format	Large set of instructions with variable formats (16-64 bits per instruction).	Small set of instructions with fixed format (32 bit).
Data transfer	Memory to memory.	Register to register.
CPU control	Most micro coded using control memory (ROM) but modern CISC use hardwired control.	Mostly hardwired without control memory.
Instruction type	Not register based instructions.	Register based instructions.
Memory access	More memory access.	Less memory access.
Clocks	Includes multi-clocks.	Includes single clock.
Instruction nature	Instructions are complex.	Instructions are reduced and simple.
Code Size	Small	Large
Example	System/360, VAX, PDP-11, Motorola 68000 family, AMD and Intel x86 CPUs.	Alpha, ARC, ARM, AVR, MIPS, PA- RISC, PIC, Power Architecture, and SPARC.

PIPELINE AND VECTOR PROCESSING

Introduction: The computational speed of the conventional computers is slow. This speed can be increased by performing simultaneous data processing. This is known as **parallel processing**. This technique is able to perform concurrent data processing to achieve faster execution. Pipeline processing is a common technique that is used to achieve parallel processing. Pipelining is a technique of decomposing a sequential process into sub-operations. Each sub-operation is executed concurrently in a special dedicated segment with all other segments. The result obtained from the computation in each segment is transferred to the next segment in the pipeline. The final result is obtained after the data have passed through all segments.

PARALLEL PROCESSING

The term parallel processing indicates that the system is able to perform several operations in a single time. Now we will elaborate the scenario, in a CPU we will be having only one Accumulator which will be storing the results obtained from the current operation.

Now if we are giving only one command such that "a+b" then the CPU performs the operation and stores the result in the accumulator. Now we are talking about parallel processing, therefore we will be issuing two instructions "a+b" and "c-d" in the same time, now if the result of "a+b" operation is stored in the accumulator, then "c-d" result cannot be stored in the accumulator in the same time.

The above problem can be solved in the following manner. Consider the registers R1 and R2 which will be storing the operands before operation and R3 is the register which will be storing the results after the operations. Now the above two instructions "a+b" and "c-d" will be done in parallel as follows

- Values of "a" and "b" are fetched in to the registers R1 and R2.
- The values of R1 and R2 will be sent into the ALU unit to perform the addition.
- The result will be stored in the Accumulator.
- When the ALU unit is performing the calculation, the next data "c" and "d" are brought into R1 and R2.
- Finally the value of Accumulator obtained from "a+b" will be transferred into the R3. Next the values of c and d from R1 and R2 will be brought into the ALU to perform the "c-d" operation.
- Since the accumulator value of the previous operation is present in R3, the result of "c-d" can be safely stored in the Accumulator.

This is the process of parallel processing of only one CPU. Consider several such CPU performing the calculations separately. This is the concept of parallel processing.

In the figure, one possible way of separating the execution unit into eight functional unit operating in parallel. The operands in the registers are applied to one of the units depending on the operation

specified by the instruction associated with the operands. The operation performed in each functional unit is indicated in each block of the diagram.

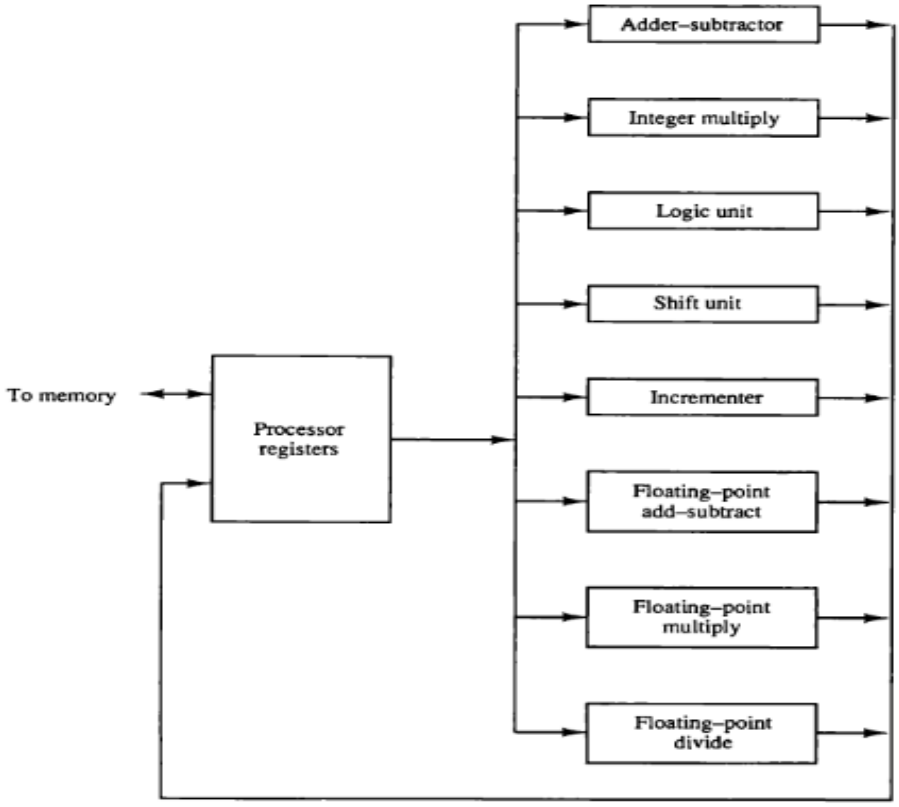


Fig: Processor with multiple functional units

If the data inside the processor registers is requesting for an arithmetic operation, then the data will be sent to the arithmetic unit and if in the same time another data is requested in the logic unit, then the data will be sent to logic unit for logical operations. Now in the same time both arithmetic operations and logical operations are executing in parallel. This is called as parallel processing.

We consider parallel processing under the following main topics:

1. Pipeline processing: Pipeline processing is an implementation technique where arithmetic suboperations or the phases of a computer instruction cycle overlap in execution.
2. Vector processing: Vector processing deals with computations involving large vectors and matrices.
3. Array processing: Array processors perform computations on large arrays of data.

PIPELINING

Pipelining is a technique where multiple instructions are overlapped during execution. Pipeline is divided into stages and these stages are connected with one another to form a pipe like structure. Instructions enter from one end and exit from another end.

Pipelining is the process of accumulating instruction from the processor through a pipeline. It allows storing and executing instructions in an ordered process. It is also known as pipeline processing.

Pipelining is a technique of decomposing a sequential process into sub operations, with each sub process being executed in a special dedicated segment that operates concurrently with all other segments. And sending the results to the next segment until the end of the processing is reached. Pipelining increases the overall instruction throughput.

- **Pipeline Structure**

Pipeline is an implementation technique where multiple instructions are overlapped in execution. The computer pipeline is divided in stages. Each stage completes a part of an instruction in parallel. The stages are connected one to next to form a pipe-instructions enter at one end, progress through the stages, and exit at the other end. The basic idea of a pipeline structure is shown in fig.

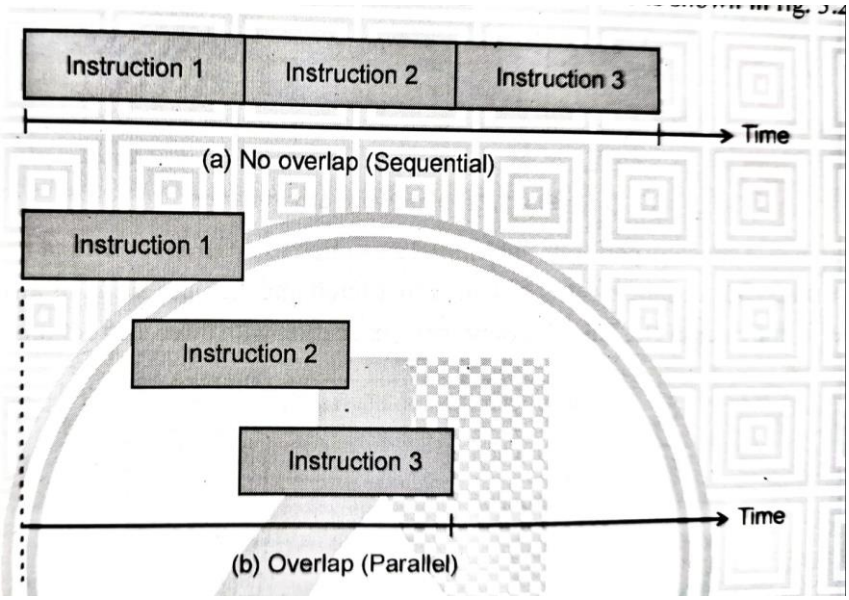


Fig: Pipeline Concept

To apply the concept of instruction execution in pipeline, it is required to break the instruction in different tasks. Each task will be executed in different processing elements of the CPU. In the simplest form, there are two distinct phases of an instruction execution.

(i) Instruction Fetch (IF)

(ii) Instruction Execution (EX)

Therefore, the processor executes a program by fetching and executing instructions, one after another. Consider that IF, and EX, represent the instruction fetch and instruction execution phases of instruction /. Then, the execution of a program consists of a sequence of fetch and execute steps is shown in fig.

IF1	EX1	IF2	EX2	IF3	EX3	…..	….	……
Instruction 1		Instruction 2		Instruction 3				

Fig: Sequential execution

Since there are two phases (**fetch and execute**) in the execution of an instruction, the CPU must contain two separate hardware units to implement the pipeline architecture. Two separate hardware units perform instruction fetching and instruction execution independently. The fetch unit, fetches the instruction from memory and stores it in an intermediate storage buffer. The hardware organization is shown in fig.

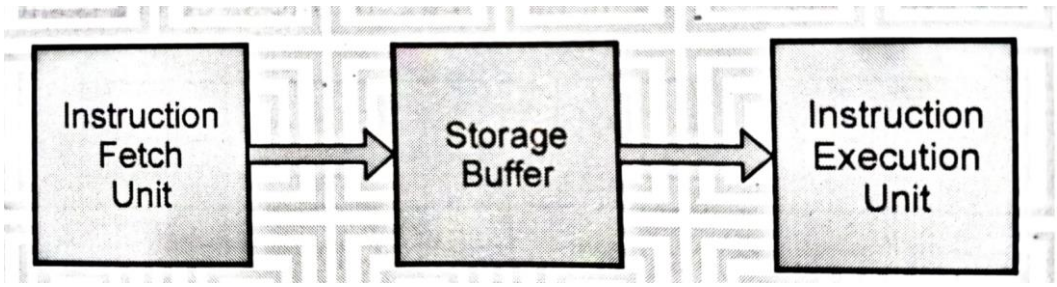


Fig: Hardware organization

The pipeline structure performs the parallel execution of different phases of instruction in different hardware units. For simplicity, it is assumed that fetch and execute steps of any instruction can be completed in one clock cycle. This idea of program execution with three instructions is shown in fig.

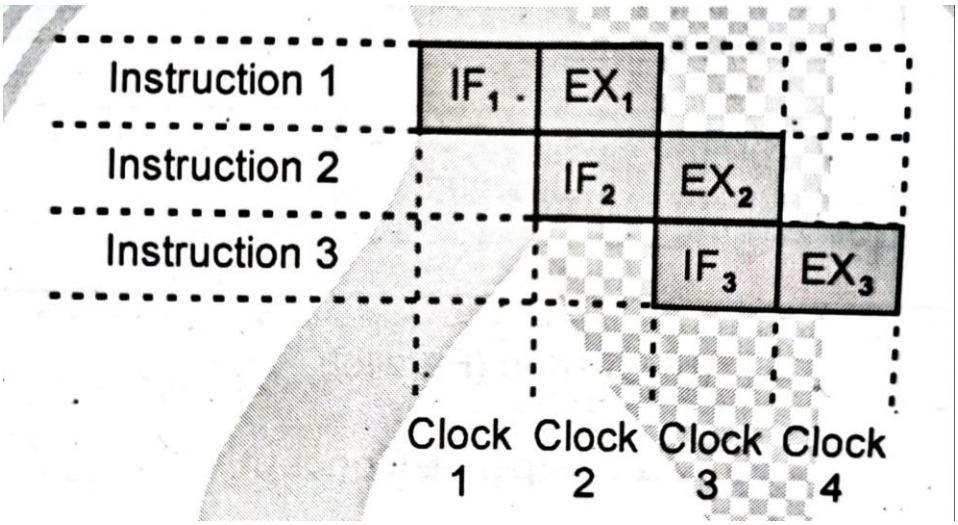


Fig: Parallel Execution with instruction fetch and execution

As shown in figure below, the program execution starts with the instruction fetch phase for instruction 1. During clock 1, the fetched instruction is also stored in intermediate buffer. Now fetch unit is free to fetch next instruction. In the clock 2, the execution unit, executes the instruction which is stored in buffer and fetch unit, fetches the next instruction parallelly. Now next instruction is stored in intermediate buffer. In this way, parallel mechanism is applied in designing the pipeline structure.

The processing of an instruction need not to be divided into only two steps. To gain further speed up, the pipeline must have more stages. Let us consider the following decomposition of the instruction execution.

(i) Instruction Fetch (IF)
(ii) Instruction Decode (ID)
(iii) Calculate Effective Address (EA)
(iv) Opcode Fetch (OF)
(v) Execution (EX)
(vi) Store Result (ST)

Now, there are six different phases. These can be performed by different hardware units. However, all the six phases do not have equal duration, there will be some waiting time involved in various stages of pipelining. Fig. shows the pipeline execution with six phases.

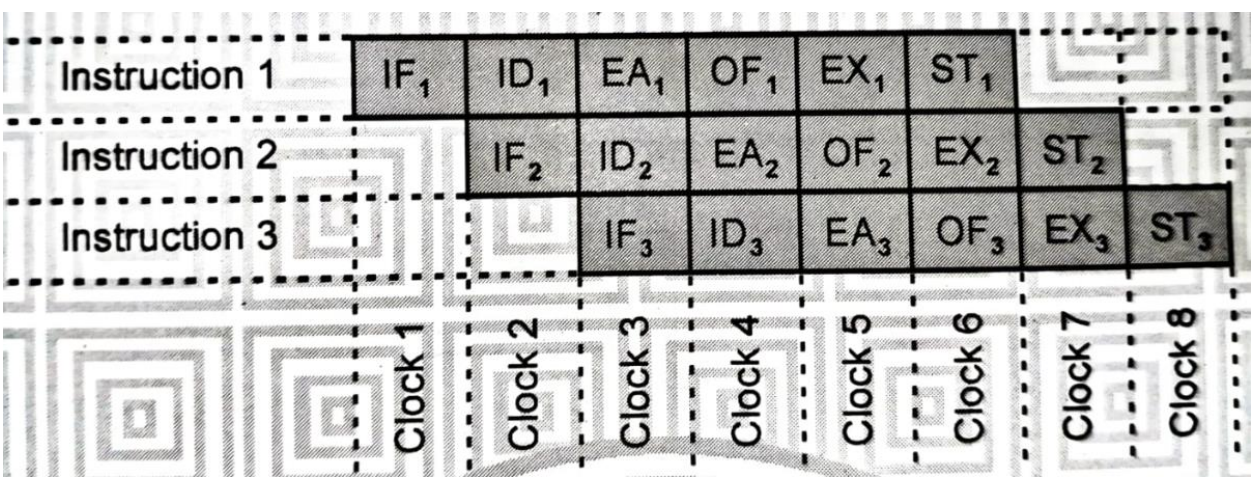


Fig. Pipeline execution with six phases

- **Types of Pipeline**

A pipeline can be visualized as a collection of processing hardware units through which the information or data flows. Each hardware unit performs partial processing dictated by the way the task is partitioned. The final result is obtained after the information or data have passed through all units.

Arithmetic and Instruction Pipeline

The design principles for arithmetic and instruction pipelines are quite different. Arithmetic pipeline divides an arithmetic operation into sub-operations for execution in the pipeline segments. It is more useful for vector processing.

An instruction pipeline operates as a stream of instructions by overlapping the fetch, decode, and execute phases of instruction cycle. It tends to be more useful for computers with simplified instruction set as in RISC architecture.

Arithmetic Pipeline

The simplest structure of pipeline may be considered as a combination of registers and combinational circuits. The register holds the data and combinational circuit performs the sub-operations in that particular segment. The arithmetic pipeline is generally implemented in very high speed computers to implement the complex arithmetic functions. This is used to perform floating point operations, multiplication of fixed point numbers and similar computations.

Example:

$$Fi = (Ai*Bi) + (Ci*Di) \qquad \text{for } i = 1, 2$$

To perform the above operation, each sub-operation is to be implemented in a segment within a pipeline. Each segment has one or two registers and a combinational circuit as shown in fig. All the registers receive and store a new data with every clock pulse. There are seven registers with two multiplier and one adder combinational circuit.

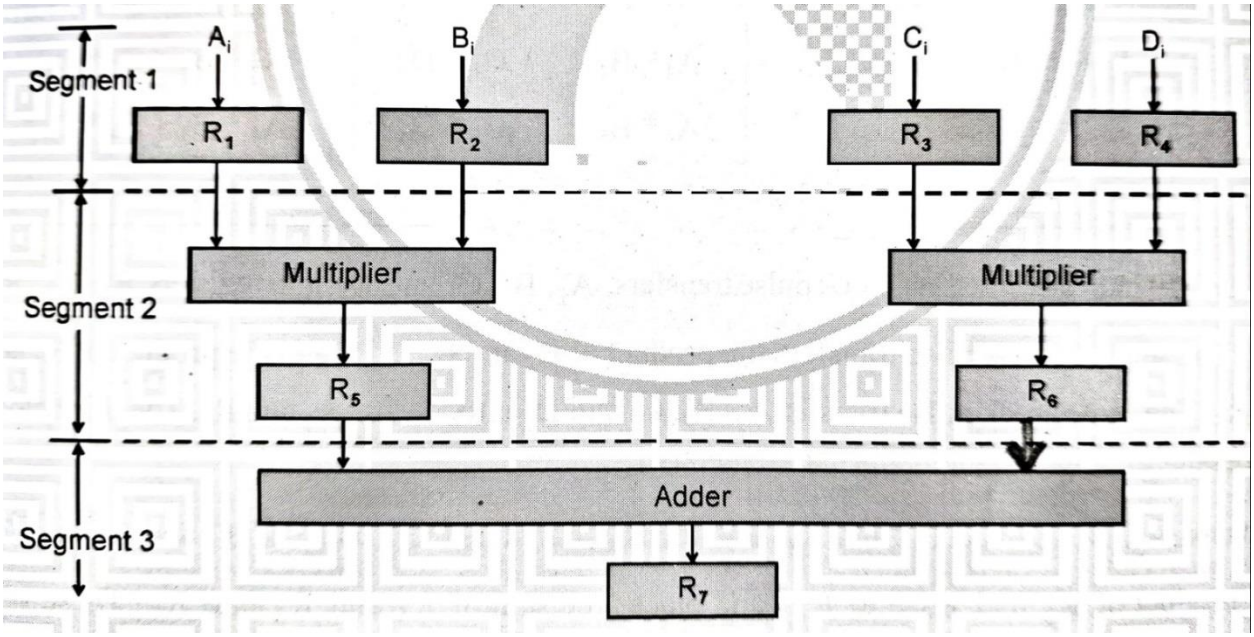


Fig: Arithmetic Pipeline (An Example)

Three segments may be considered as shown in Fig. All these segments performs their functions separately. Once the segment 1 completes its operation, it passes its output to the segment 2 and segment 2 passes its output to the segment 3 after completing its own operation.

Table: Sub-operations performed in each segment in arithmetic pipeline

Segment	Register Transfer Language	Function
Segment 1	$R_1 \leftarrow A_i$ $R_2 \leftarrow B_i$, $R_3 \leftarrow C_i$, $R_4 \leftarrow D_i$	Input data
Segment 2	$R_5 \leftarrow R_1 * R_2$, $R_6 \leftarrow R_3 * R_4$	Multiplication
Segment 3	$R_7 \leftarrow R_5 + R_6$	Addition

All the seven registers (R1, to R7) are loaded with new data on occurrence of clock pulse. The effect of each clock pulse is given in table below.

Table: Content of registers in pipeline example

Clock Plse	Segment 1				Segment 2		Segment 3
	R_1	R_2	R_3	R_4	R_5	R_6	R_7
1	A_1	B_1	C_1	D_1	–	–	–
2	A_2	B_2	C_2	D_2	$A_1 * B_1$	$C_1 * D_1$	–
3	A_3	B_3	C_3	D_3	$A_2 * B_2$	$C_2 * D_2$	$A_1 * B_1 + C_1 * D_1$
4	A_4	B_4	C_4	D_4	$A_3 * B_3$	$C_3 * D_3$	$A_2 * B_2 + C_2 * D_2$
5	A_5	B_5	C_5	D_5	$A_4 * B_4$	$C_4 * D_4$	$A_3 * B_3 + C_3 * D_3$
6	–	–	–	–	$A_5 * B_5$	$C_5 * D_5$	$A_4 * B_4 + C_4 * D_4$
7	–	–	–	–	–	–	$A_5 * B_5 + C_5 * D_5$

Table indicates, the first clock pulse transfers, A1, B1, C1, and D1 into registers R1, R2, R3 and R3.

The second clock pulse transfers the product of R1 and R2 in R5, and the product of R3 and R4 into R6. The same clock pulse transfer A2, B2, C2 and D2 into R1, R2, R3 and R4.

The third clock pulse operates an all three segments simultaneously. It places A3, B3, C3, and D3, into R1, R2, R3, R4, transfers the product of R1 and R2 into R5 and the product of R3 and R4 into R6, and places the sum of R5 and R6 into R7. These sub- operations show that three clock pulses are taken to fill the pipe and to retrieve the first output from R7. After that, each clock produces a

new output and moves the data one step down in pipeline. It is important to note that when no input data are available, the clock must continue until the last output emerges out of the pipeline.

Instruction Pipeline

Arithmetic Pipeline include steps such as fetching and decoding the instruction, calculating the effective address, fetching the operands, executing the instruction and finally storing the result at the desired memory location. These operations were performed in the pre-specified sequence and make the system slower.

An instruction pipeline increases the performance of a processor by overlapping the processing of several different sub operations. Often, this is done by dividing the instruction execution process into several segments.

The design of an instruction pipeline will be efficient if the instruction cycle is divided into segments of equal duration.

- **Phases of Instruction Execution**

In the simplest way there are only two phases to process any instruction:

(i) Instruction fetch

(ii) Execution.

But computers with complex instructions require more phases in addition to the instruction fetch and execution. In most of the cases, the computer needs to process each instruction with the following phases:

(i) Instruction Fetch (IF): The processor fetches or retrieves the instruction code from cache or main memory.
(ii) Instruction Decoding (ID): This phase includes the decoding of instruction. Effective address of the operands is also calculated if required, in this phase,
(iii) Operand Fetch (OF): Retrieval of any required operands from memory is done in this phase.
(iv) Execution (EX): In this phase execution of fetched instruction takes place.
(v) Store Result (ST): This phase stores the result in the proper place.

Fig. shows the instruction cycle in the processor with five phases.

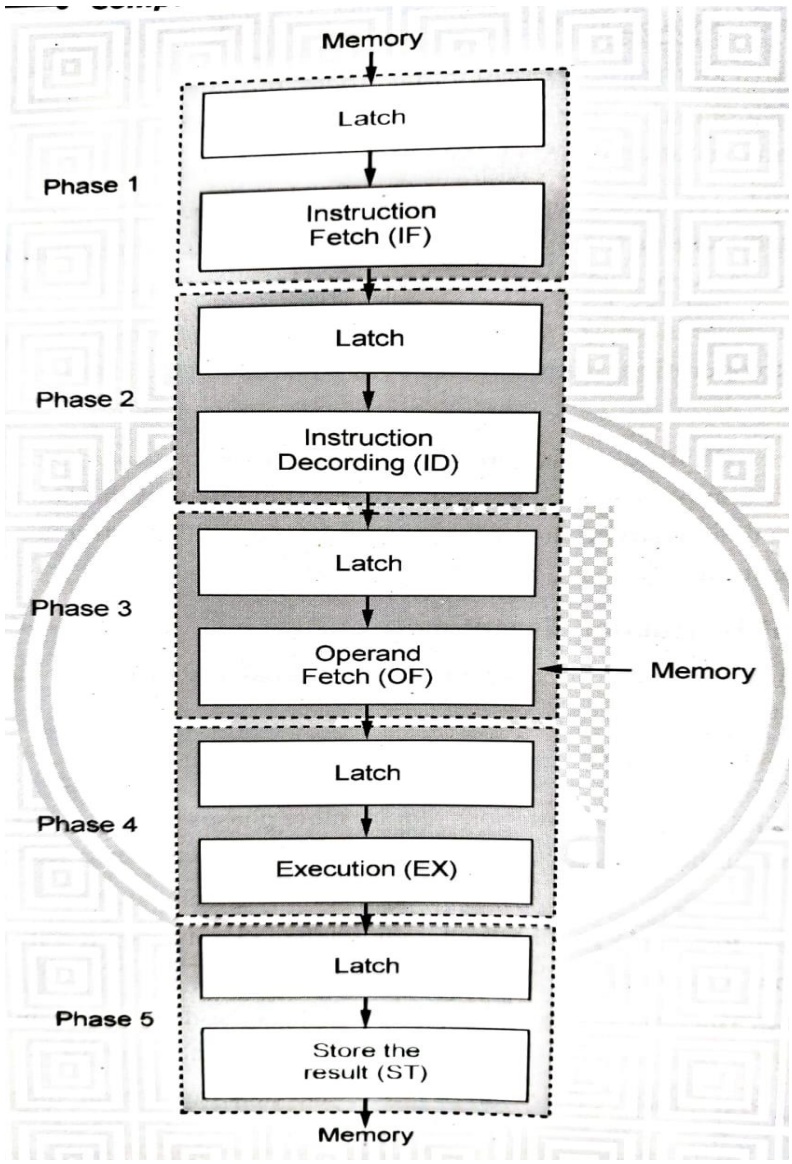


Fig. Instruction cycle in the processor with five phases.

Consider an example of five instructions program. It is assumed that the processor has separate instruction and data memories. During the first clock pulse, instruction i_1, is fetched from memory. Within second cycle, instruction i_1, is decoded and instruction i_2 is fetched from memory. This process continues until all the instructions are executed.

Table: Execution of instruction pipeline

Instruction	CLOCK PULSE								
	1st	2nd	3rd	4th	5th	6th	7th	8th	9th
i_1	IF	ID	OF	EX	ST				
i_2	–	IF	ID	OF	EX	ST			
i_3	–	–	IF	ID	OF	EX	ST		
i_4	–	–	–	IF	ID	OF	EX	ST	
i_5	–	–	–	–	IF	ID	OF	EX	ST

Advantages of Pipelining

1. The cycle time of the processor is reduced.
2. It increases the throughput of the system
3. It makes the system reliable.

Disadvantages of Pipelining

1. The design of pipelined processor is complex and costly to manufacture.
2. The instruction latency is more.

Pipeline Parameter

The pipeline technique provides faster operation over a serial processing. However, the maximum theoretical speed is never fully achieved. Different segments may take different times to complete their sub-operation. This causes to wait one segment to other, to complete the sub-operation. This results in reduction efficiency of the pipeline architecture. There are various parameters which define the efficiency of the pipeline.

- **Speed-up**

The speed-up of a pipeline processing over an equivalent non pipeline processing is defined as the ratio.

$$S = \frac{nt_n}{(k + n - 1)t_p}$$

n = number of sub-operations
k = number of segments in pipeline.
tp = clock cycle time to complete each sub-operation in pipeline process.
tn = clock cycle time to complete operation in nonpipeline process.

Case-I: If the number of sub-operations are increased, n becomes much larger than k-1. It means k+n-1=n, Therefore

$$S = \frac{nt_n}{(k+n-1)t_p} = \frac{nt_n}{nt_p} = \frac{t_n}{t_p}$$

$$S = \frac{t_n}{t_p}$$

Case-II: If the time taken to process a sub-operation is same in the pipeline and non-pipeline processes, it can be written as

$$1 \times t_n = k \times t_p$$

Since there is only one segment in non-pipeline process.

Substitute this assumption to the speed-up formula of case-I, we get

$$S = \frac{t_n}{t_p} = \frac{k \times t_p}{t_p} = k$$

This shows that the maximum speed-up that a pipeline can achieve is equal to number of segments (k).

Note: A k-segment pipeline can be expected to equal in performance of k copies of an equivalent nonpipeline under equal operating conditions.

Example: Determine the number of clock cycles that it takes to process 200 sub-operations in a six segment pipeline.

Solution: Given that Number of segments (k) = 6

Number of sub operations (n) = 200

Therefore, number of clock cycles required = k+n-1

=6+200-1=205 cycles.

Example: A non-pipeline system takes 50 ns to process a task. The same task can be processed in a six-segment pipeline with a clock cycle of 10 ns. Determine the speed-up ratio of pipeline for 100 task. What is the maximum speed-up that can be achieved?

Solution: Given that

k=6, n=100, tp=10 n sec, tn=50 n sec.

Therefore,

$$S = \frac{nt_n}{(k + n - 1)t_p}$$

$$S = \frac{100 \times 50}{(6 + 100 - 1) \times 10} = 4.76$$

And maximum speed-up

$$S = \frac{t_n}{t_p} = \frac{50}{10} = 5$$

Pipeline Efficiency:

The efficiency of a pipeline architecture is defined as the ability of the pipeline to handle the parallel computation of data. To make pipelines work effectively three simple modifications are added to the internal architecture of pipeline.

(i) Uniform Instruction Length: All instruction have a uniform byte length. This means loading the instructions is always the same, and decoding the instructions is straight forward.

(ii) Simple Addressing Modes: Only simple addressing modes are allowed.

(iii) Simple Load/Store Modes: Only simple load and store commands are allowed. There are no complicated multi-cycle load or store commands in the processor.

All these modifications are for the sake of efficiency. This has no real effect on the types of programs allowed in a high level language. It only impacts how the compiler translates the program into machine code.

Throughput

Throughput of a pipeline architecture is defined as the amount of processing that can be accomplished during a given interval of time. In other words, it can be considered as the number instructions that can be executed in a unit of time.

Throughput of a pipeline architecture can be increased by increasing the number of segments, but it also increases the hardware cost and with it, the cost of system also increases. However, technological developments have reduced the hardware costs to the point where pipeline architecture with higher throughput are economically feasible.

In general, it may be thinking that the pipeline throughput should always be 1 instruction/clock because pipelined processor completes a new instruction at the end of each clock cycle. When pipeline is full, the pipelined processor has a throughput 1 instruction/clock.

There are three types of throughput.

(i) Instruction Throughput: It is the number of instructions that the processor completes executing on each clock cycle. It is also known as instructions per clock (IPC).

(ii) Maximum Theoretical Instruction Throughput: It is the theoretical maximum number of instructions that the processor can complete executing on each clock cycle. For the simple kinds of pipelined and non-pipelined processors, this is always 1 instructions/clock (1 IPC).

(iii) Average Instruction Throughput: This is the average number of instructions per clock (IPC) that the processor has actually completed over a certain number of cycles.

RISC PIPELINE

The ability to use the instruction pipelining concept in the RISC architecture is very efficient. The simplicity of the instruction set can be utilized to implement an instruction pipeline using a small number of sub operations, with each being executed in one clock cycle. Due to fixed length instruction format, the decoding of the operation can occur at the same time as the register selection. Since the arithmetic, logic and shift operations are done on register basis, there is no need for extra fetching or effective address decoding steps to perform the operation. So pipelining concept can be effectively used in this scenario.

Therefore the total operations can be categorized as one segment will be fetching the instruction from program memory, the other segment executes the instruction in the ALU and the third segment may be used to store the result of the ALU operation in a destination register. The data transfer instructions in RISC are limited to only Load and Store instructions. To prevent conflicts in data transfer, we will be using two separate buses one for storing the instructions and other for storing the data. The two memories can some time operate at the same speed as the CPU clock and are referred to as cache memories.

RISC pipelining is the ability to execute instruction at the rate of one per clock cycle and it support given by the compiler that translates the high level language into machine language program.

Example of three segment instruction pipeline:

We want to perform an operation in which there is some arithmetic, logic or shift operations. Therefore as per the instruction cycle, we will be having the following steps:

I: Instruction Fetch

A: ALU Operation

E: Execute Instruction.

The **I** (IF) segment will be fetching the instruction from program memory. The instruction is decoded and an ALU operation is performed in the **A** (ALU) segment. In the A segment the ALU operation instruction will be fetched and the effective address will be retrieved and finally in the **E** (EI) segment the instruction will be executed.

Delayed Load:

Consider the following instructions:

1. LOAD: R1←M[address 1]
2. LOAD: R2 ←M[address 2]
3. ADD: R3← R1 + R2
4. STORE: M[address 3] ←R3

The below tables will be showing the pipelining concept with the data conflict and without data conflict.

Clock cycles :	1	2	3	4	5	6
1. Load R1	I	A	E			
2. Load R2		I	A	E		
3. Add R1 + R2			I	A	E	
4. Store R3				I	A	E

(a) Pipeline timing with data conflict

Clock cycles :	1	2	3	4	5	6	7
1. Load R1	I	A	E				
2. Load R2		I	A	E			
3. No-operation			I	A	E		
4. Add R1 + R2				I	A	E	
5. Store R3					I	A	E

(b) Pipeline timing with delayed load

Fig: Three-segment pipeline timing

Delayed Branch:

Branch Instruction delays the pipeline operation until the instruction at the branch address is fetched. The method used in most RISC processors is to rely on the compiler to redefine the branches so that they effect at the proper time in the pipeline. This method is referred to as delayed branch.

An example of delayed branch is show below:

Clock cycles :	1	2	3	4	5	6	7	8	9	10
1. Load	I	A	E							
2. Increment		I	A	E						
3. Add			I	A	E					
4. Subtract				I	A	E				
5. Branch to X					I	A	E			
6. No-operation						I	A	E		
7. No-operation							I	A	E	
8. Instruction in X								I	A	E

(a) Using no-operation instructions

Clock cycles :	1	2	3	4	5	6	7	8
1. Load	I	A	E					
2. Increment		I	A	E				
3. Branch to X			I	A	E			
4. Add				I	A	E		
5. Subtract					I	A	E	
6. Instruction in X						I	A	E

(b) Rearranging the instructions

Fig: Example of delayed branch

The program for this example consist of five instruction-

- Load from memory to R1
- Increment R2
- Add R3 to R4
- Subtract R5 from R6
- Branch to address X

In Fig. (a) The compiler inserts two no-op instructions after the branch. The branch address X is transferred to PC in clock cycle 7. The fetching of the instruction at X is delayed by two clock cycles by the no-operation instructions. The instruction at X starts the fetch phase at clock cycle 8 after the program counter PC has been updated.

The program in Fig. (b) is rearranged by placing the add and subtract instructions after the branch instruction instead of before as in the original program. Inspection of the pipeline timing shows that PC is updated to the value of X in clock cycle 5, but the add and subtract instructions are fetched from memory and executed in the proper sequence.

VECTOR PROCESSING

The need to increase computational power is a never-ending requirement. In scientific and research areas, the computational involved are quite extensive and hence high power computers are the must.

A vector processor is one whose instructions operate on vectors rather than scalar (single data) values. A vector is a one-dimensional array of numbers.

Computers with vector processing capabilities are in demand in specialized applications. The following application areas when vector processing is used.

- Long-range weather forecasting

- Radar and signal processing
- Medical diagnosis
- Aerodynamics and space fight simulations
- Artificial intelligence and expert systems
- Mapping the human genome
- Petroleum explorations
- Seismic data analysis
- Wind tunnel computational
- Image processing

The areas like structural engineering, petroleum exploration, aerodynamics, hydrodynamics, nuclear research, tomography, VLSI design, AI can have data in the form of matrices which suits vector processors to process it at a high speed.

A block diagram of a modern multiple pipeline vector computer is shown below in fig:

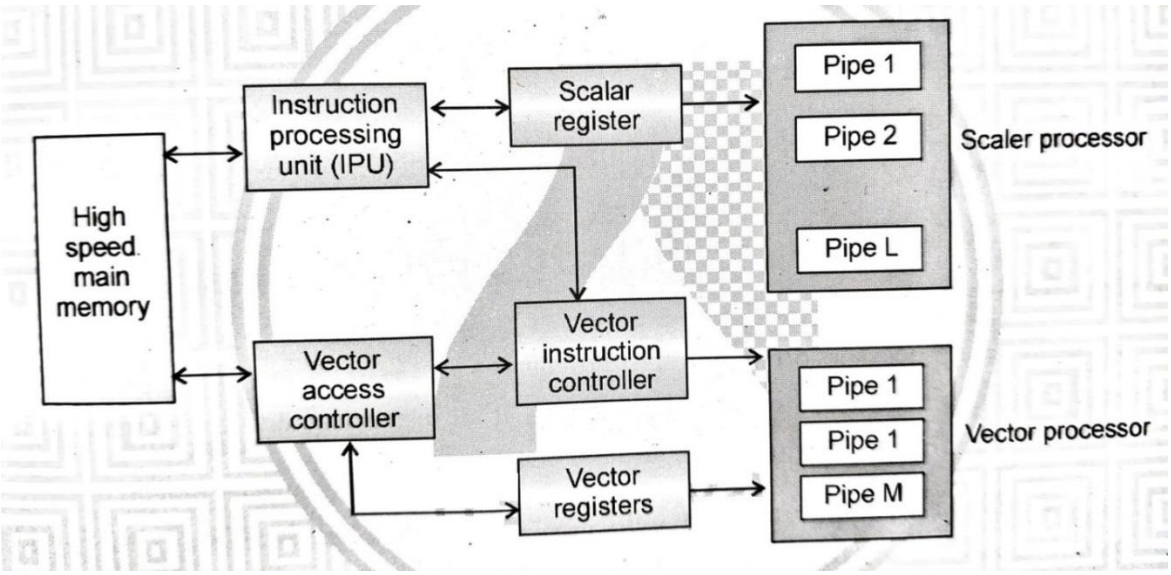


Fig: Vector computer

ARRAY PROCESSORS

This architecture includes a number of processors (say 64 by 64) working simultaneously, each handling one element of the array, so that a single operation can apply to all elements of the array in parallel. To obtain the same effect in a conventional processor, the operation must be applied to each element of the array sequentially and so consequently much more slowly.

An array processor may be built as a self-contained unit attached to a main computer via an I/O port or internal bus; alternatively, it may be a distributed array processor where the processing elements are distributed throughout, and closely linked to, a section of the computer's memory.

Types of Array Processors:

There are basically two types of array processors:

1. Attached Array Processors
2. SIMD Array Processors

- **Attached Array Processors**

An attached array processor is a processor which is attached to a general purpose computer and its purpose is to enhance and improve the performance of that computer in numerical computational tasks. It achieves high performance by means of parallel processing with multiple functional units.

A host computer is a general purpose commercial computer and the attached processor is a back end machine driven by the host computer.

The array processor connected through an input-output controller to the computer and the computer treats it like an external interface. The data for the attached processor are transferred from main memory to a local memory through a high-speed bus.

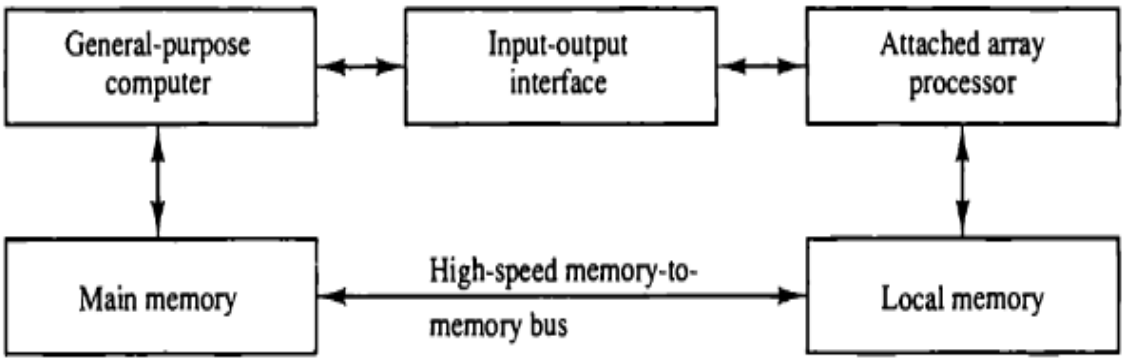


Fig: Attached Array Processors

- **SIMD Array Processors**

SIMD (Single instruction and multiple data) is the organization of a single computer containing multiple processors operating in parallel. The processing units are made to operate under the control of a common control unit, thus providing a **single instruction stream and multiple data streams**. It contain a set of identical processing elements (PEs), each having a local memory M. Each processor element includes an ALU, a floating-point arithmetic unit, and working registers.

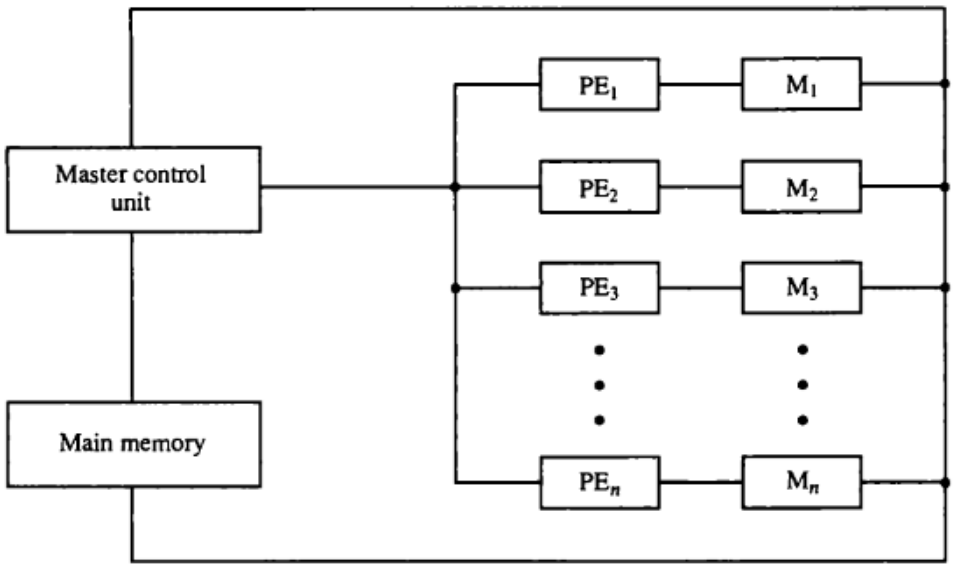


Fig: SIMD array processor organization

The master control unit controls the operations in the processor elements. The main memory is used for storage of the program. The function of the master control unit is to decode the instructions and determine how the instruction is to be executed.

Scalar and program control instruction are directly executed within the master control unit. Vector instructions are broadcast to all PEs simultaneously.

Each PE uses operands stored in its local memory. Vector operands are distributed to the local memories prior to the parallel execution of the instruction.

The best known SIMD array processor is the ILLIAC IV (**Ill**inois **A**utomatic **C**omputer) computer developed by the Burroughs corps. SIMD processors are highly specialized computers.

They are only suitable for numerical problems that can be expressed in vector or matrix form.

Advantages of Array Processor

1. Array processors increases the overall instruction processing speed.
2. As most of the Array processors operates asynchronously from the host CPU, hence it improves the overall capacity of the system.
3. Array Processors has its own local memory, hence providing extra memory for systems with low memory.